Searching Algorithms

Array is basic data structures used to store information. We may wish to search, insert or delete records in a database based on a key value. This section examines the performance of these operations.

1. Sequential Search

The simplest search algorithm is linear search. In linear search, we look at each item in the list in turn,quitting once we find an item that matches the search term or once we've reached the end of the list. Our"return value" is the index at which the search term was found, or some indicator that the search term wasnot found in the list.

Figure 1-1 shows an array, seven elements long, containing numeric values. To search the array sequentially,, we may use the algorithm in Figure 1-2. The maximum number of comparisons is 7, and occurs when the key we are searching for is in A[6].

1.1 Algorithm for linear search

```
for (each item in list)
{
compare search term to current item
if match,
save index of matching item
break
}
return index of matching item, or -1 if item not found
```

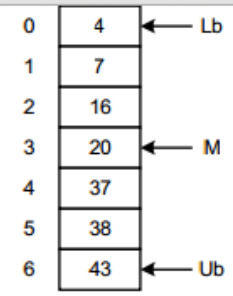


Figure 1-1: An Array

```
int function SequentialSearch (Array A , int Lb , int Ub , int Key );
    begin
    for i = Lb to Ub do
        if A [ i ] = Key then
            return i ;
        return –1;
    end;
```

Figure 1-2: Sequential Search

1.2 C++ implementation of linear search

```
protectedint[] a;

publicintlinearSearch (inttarget)

{

for(int n = 0; n <a.length; n++)

 {

if(a[n] == target)

   {

return n;

   }

return-1;

 }

}
```

1.3 Performance of linear search

When comparing search algorithms, we only look at the number of comparisons, since we don't swap anyvalues while searching. Often, when comparing performance, we look at three cases:

• Best case: What is the fewest number of comparisons necessary to find an item?

• Worst case: What is the most number of comparisons necessary to find an item?

• Average case: On average, how many comparisons does it take to find an item in the list?

For linear search, our cases look like this:

• Best case: The best case occurs when the search term is in the first slot in the array. The number ofcomparisons in this case is 1.

• Worst case: The worst case occurs when the search term is in the last slot in the array, or is not inthe array. The number of comparisons in this case is equal to the size of the array. If our array has Nitems, then it takes N comparisons in the worst case.

• Average case: On average, the search term will be somewhere in the middle of the array. The numberof comparisons in this case is approximately N/2.

In both the worst case and the average case, the number of comparisons is proportional to the number ofitems in the array, N. Thus, we say in these two cases that the number of comparisons is order N, or O(N)for short. For the best case, we say the number of comparisons is order 1, or O(1) for short.

2. Binary search

Linear search works well in many cases, particularly if we don't know if our list is in order. Its one drawbackis that it can be slow. If N, the number of items in our list, is 1,000,000, then it can take a long time onaverage to find the search term in the list (on average, it will take 500,000 comparisons).

What if our list is already in order? Think about looking up a name in the phone book. The names in thephone book are ordered alphabetically. Does it make sense, then, to look for "Sanjay Kumar" by startingat the beginning and looking at each name in turn? No! It makes more sense to exploit the ordering of thenames, start our search somewhere near the K's, and refine the search from there.



Binary search exploits the ordering of a list. The idea behind binary search is that each time we makea comparison, we eliminate half of the list, until we either find the search term or determine that the termis not in the list. We do this by looking at the middle item in the list, and determining if our search termis higher or lower than the middle item. If it's lower, we eliminate the upper half of the list and repeat oursearch starting at the point halfway between the first item and the middle item. If it's higher, we eliminatethe lower half of the list and repeat our search starting at the point halfway between the middle item andthe last item.

2.1 Algorithm for binary search

```
set first = 1, last = N, mid = N/2

while (item not found and first < last)

{

        compare search term to item at mid

        if match

save index

break

        else if search term is less than item at mid,

set last = mid-1

else

set first = mid+1

set mid = (first+last)/2

}

return index of matching item, or -1 if not found
```

Using Figure1-1

```
int function BinarySearch (Array A , int Lb , int Ub , int Key );
    begin
    do forever
        M = ( Lb + Ub )/2;
        if ( Key < A[M]) then
            Ub = M – 1;
        else if (Key > A[M]) then
            Lb = M + 1;
        else
            return M ;
        if (Lb > Ub) then
            return –1;
    end;
```

Figure 1-3: Binary Search

2.2C++ implementation of binary Search

```
publicintbinarySearch(int key, int list[],int low, int high) {
int index = -1;
int mid;
while (high >= low)
{
mid = (high + low)/2;// calculate the midpoint of the current array
if (item < list[mid]) // value is in lower half, if at all
{ high = mid - 1;
}
else if (item > list[mid]) // value is in upper half, if at all
{
low = mid + 1;
}
else
{
```

```
index = mid;
break;
}
}
return index;
}
```

2.3 Performance of binary search

- The best case for binary search still occurs when we find the search term on the first try. In this case, the search term would be in the middle of the list. As with linear search, the best case for binary search is O(1),since it takes exactly one comparison to find the search term in the list.
- The worst case for binary search occurs when the search term is not in the list, or when the search termis one item away from the middle of the list, or when the search term is the first or last item in the list. Howmany comparisons does the worst case take? To determine this, let's look at a few examples.

Suppose we have a list of four integers: {1, 4, 5, 6}. We want to find 2 in the list. According to thealgorithm, we start at the second item in the list, which is 4.2 Our search term, 2, is less than 4, so we throwout the last three items in the list and concentrate our search on the first item on the list, 1. We compare 2to 1, and find that 2 is greater than 1. At this point, there are no more items left to search, so we determinethat 2 is not in the list. It took two comparisons to determine that 2 is not in the list.Now suppose we have a list of 8 integers: {1, 4, 5, 6, 9, 12, 14, 16}. We want to find 9 in the list. Again,we find the item at the midpoint of the list, which is 6. We compare 6 to 9, find that 9 is greater than 6,and thus concentrate our search on the upper half of the list: {9, 12, 14, 16}. We find the new midpointitem, 12, and compare 12 to 9. 9 is less than 12, so we concentrate our search on the lower half of this list(9). Finally, we compare 9 to 9, find that they are equal, and thus have found our search term at index 4 inthe list. It took three comparisons to find the search term.If we look at a list that has 16 items, or 32 items, we find that in the worst case it takes 4 and 5comparisons, respectively, to either find the search term or determine that the search term is not in thelist. In all of these examples, the number of comparisons is log2 N.3 This is much less than the number ofcomparisons required in the worst case for linear search! In general, the worst case for binary search is orderlog N, or O(logN).The average case occurs when the search term is anywhere else in the list. The number of comparisonsis roughly the same as for the worst case, so it also is O(logN).

CS214: Data Structure

Lab/Section 3– Searching

In general, anytime an algorithm involves dividing a list in half, the number of operations is O(log N).

Section problems

Problem 1 :

Create a class OrderedArray that contain the following attributes and functions:

```
template<class T>
class MyVector
{
T* arr;
int  maxSize, currSize;
bool resize(int newSize);
////////////////////
MyVector (int mSize=100);
int recursiveSeqSearch (T target,int i);
bool addElement(T v);
void printDetails();
};
```

CS214: Data Structure

Lab/Section 3– Searching



CairoUniversity,

Lab problems

Problem 1: Message encoder and decoder

Using the table below, implement binary search algorithm to encode a message" This is a secret message"

Input: thisisasecretmessage**output**: =*(,(,#,%@!%=)%,,#&%

Then implement sequential search algorithm to decode the above message.

Letter	Code
a	#
c	@
e	%
g	&
h	*
i	(
m	)
r	!
s	,
t	=

Problem 2: Lottery Winners

A lottery ticket buyer purchases ten tickets a week, always playing the same ten five-digit "lucky" combinations. Write a program that initializes an array with these numbers and then lets the player enter this week's winning five-digit number. The program should perform a linear search through the list of the player's numbers and report whether or not one of the tickets is a winner this week. Here are the numbers:

13579 26791 26792 33445 55555

62483 77777 79422 85647 93121

Problem 3: Lottery Winners Modification

Modify the program you wrote for Programming Challenge 2 (Lottery Winners) so it performsa binary search instead of a linear search.

Problem 4:

Write a program that has an array of at least 20 integers. It should call a function that uses the linear search algorithm to locate one of the values. The function should keep a count of the number of comparisons it makes until it finds the value. The program then should call a function that uses the binary search algorithm to locate the same value. It should also keep count of the number of comparisons it makes. Display these values on the screen.